



PROGRESSFIT

2º DAW



12 DE JUNIO DE 2026
ÁNGEL DAVID ALCÁNTARA GARABITO
71783400D

Contenido

1. Resumen	3
1.1. Palabras Claves	3
2. Introducción.....	4
3. Objetivos	5
4. Análisis del contexto y estado del arte.....	6
4.1 Análisis del contexto.....	6
4.2. Estado del arte	6
4.3. Innovación	7
4.4. DAFO.....	8
5. Planificación	9
5.1. Temporalización del proyecto	9
5.2. Recursos humanos y materiales	11
5.3. Presupuesto y financiación	11
6. Análisis del sistema	12
6.1. Definición de requisitos	12
6.2. Identificación de los subsistemas.....	14
6.3. Análisis de casos de uso	16
6.4. Análisis de interfaces.....	19
6.5. Especificación del plan de pruebas	21
7. Diseño del sistema	23
7.1 Arquitectura del sistema	23
7.2. Diseño de clases.....	25
7.3. Diseño de la base de datos	27
7.4 Otros diagramas.....	30
7.5 Diseño de interfaces.....	32
8. Implementación	46
8.1. Lenguajes, frameworks y tecnologías.....	46
8.2. Problemas encontrados y soluciones	48
9. Desarrollo de las pruebas	49
9.1. Pruebas de código.....	49

9.2 Pruebas de usabilidad	49
9.3 Pruebas de accesibilidad	50
10.Manuales del sistema	52
10.1.Manual de despliegue.....	52
10.2. Manual de usuario	55
11.Conclusiones	57
11.1.Desarrollo del proyecto.....	57
11.2 Futuras ampliaciones	58
12.Bibliografía	59

1. Resumen

ProgressFit es una **aplicación web full-stack** diseñada para que las personas que practican entrenamiento de fuerza puedan **registrar y organizar** sus rutinas de **gimnasio** de forma digital.

La idea surge de una práctica habitual entre quienes van al gimnasio: **anotar sus ejercicios, series, repeticiones y pesos en una libreta o en notas del móvil**, un método desordenado que dificulta consultar el historial y, sobre todo, comprobar si se está progresando.

La aplicación permite al usuario **registrar** sus entrenamientos, llevar un control de su peso corporal y visualizar su evolución mediante **gráficas y estadísticas**. Además, calcula indicadores útiles como el **Índice de Masa Corporal** y una estimación de **las calorías diarias** recomendadas según su objetivo.

De esta forma, el usuario dispone en un único lugar de toda la información necesaria para entrenar de manera **planificada** y medir objetivamente su rendimiento a lo largo del tiempo, sustituyendo el método tradicional en papel por una herramienta accesible, ordenada y visual.

1.1. Palabras Claves

Aplicación web, entrenamiento, fitness, gimnasio, seguimiento de progreso, registro de entrenamientos, React, Spring Boot, gestión de rutinas, IMC, estadísticas deportivas.

2. Introducción

El **ejercicio físico** y, en concreto, el entrenamiento de fuerza en el gimnasio se ha convertido en una actividad cada vez más extendida. Cualquier persona que entrene de forma **constante** sabe que progresar no es cuestión de azar, sino de **planificación**: es necesario llevar un control de qué ejercicios se realizan, con cuántas series y repeticiones, qué peso se levanta en cada momento y cómo evoluciona todo ello con el paso de las semanas.

A raíz de mi propia experiencia entrenando y de observar a varias personas de mi entorno, detecté un problema común: la mayoría apunta sus entrenamientos en una libreta de papel o en las notas del teléfono. Este método, aunque funcional, presenta varios inconvenientes. La información queda dispersa y desordenada, es difícil consultar entrenamientos antiguos, y resulta prácticamente imposible ver de un vistazo si se está mejorando o estancado. En definitiva, se recogen muchos datos, pero no se aprovechan.

De esta necesidad nace **ProgressFit**. La idea es trasladar ese cuaderno de entrenamiento al entorno digital, pero dándole un valor añadido: no solo almacenar **los datos**, sino **organizarlos** y convertirlos en información útil mediante **estadísticas y gráficas** que muestren la **evolución del usuario**. El objetivo es que cualquier persona pueda registrar sus sesiones de forma rápida y, sobre todo, comprobar de manera clara y visual si su rendimiento mejora con el tiempo.

Se trata de un proyecto de desarrollo de una aplicación propia, no vinculado a ninguna empresa, concebido como herramienta de uso personal para deportistas aficionados que quieran entrenar de forma más organizada y medir sus resultados.

3. Objetivos

Objetivo general

Desarrollar una aplicación web que permita a los usuarios registrar sus entrenamientos de gimnasio y realizar un seguimiento de su progreso físico de forma organizada, sustituyendo los métodos tradicionales en papel por una herramienta digital, accesible y visual.

Objetivos específicos

1. Implementar un sistema de registro e inicio de sesión de usuarios mediante autenticación segura, que proteja el acceso a los datos personales de cada usuario.
2. Permitir al usuario crear y registrar sus entrenamientos, indicando los ejercicios realizados junto con sus series, repeticiones y peso levantado.
3. Ofrecer un catálogo de ejercicios organizados por grupos musculares, que sirva de base para construir los entrenamientos.
4. Permitir al usuario llevar un control de su peso corporal a lo largo del tiempo mediante registros periódicos.
5. Calcular y mostrar indicadores físicos de interés para el usuario, como el Índice de Masa Corporal (IMC) y una estimación de las calorías diarias recomendadas según su objetivo.
6. Visualizar la evolución del usuario mediante gráficas y estadísticas que reflejen su progreso en peso, fuerza y frecuencia de entrenamiento.
7. Proporcionar un perfil de usuario donde se puedan consultar y modificar los datos personales y los objetivos.
8. Incorporar un panel de administración que permita gestionar usuarios, así como el catálogo de músculos y ejercicios de la aplicación.
9. Ofrecer un sistema de recuperación de contraseña que permita al usuario restablecer su acceso en caso de olvido.
10. Diseñar una interfaz adaptada a dispositivos móviles, dado que es el dispositivo más habitual de uso en un entorno de gimnasio.

4. Análisis del contexto y estado del arte

4.1 Análisis del contexto

ProgressFit se enmarca en el sector de las **aplicaciones de fitness y salud**, concretamente en el nicho de las aplicaciones de registro y seguimiento de entrenamientos de fuerza (conocidas en inglés como workout trackers o gym logs). Se trata de un nicho consolidado y en crecimiento, impulsado por el auge del entrenamiento en gimnasio y por la tendencia general a digitalizar y cuantificar la actividad física.

Las características de este nicho son las siguientes: su público objetivo son personas que practican entrenamiento de fuerza de forma regular, desde principiantes que buscan orientación hasta usuarios avanzados que quieren un control detallado de su rendimiento.

El uso de estas aplicaciones se produce mayoritariamente desde el teléfono móvil, ya que es el dispositivo que el usuario lleva consigo al gimnasio. La necesidad común que cubren todas ellas es la misma: registrar qué se entrena y poder analizar la evolución a lo largo del tiempo para asegurar el progreso.

4.2. Estado del arte

En la actualidad existen varias aplicaciones consolidadas que compiten en este nicho y que constituyen la competencia directa de ProgressFit:

Hevy

Es una de las aplicaciones de referencia, con más de 10 millones de usuarios. Destaca por una interfaz moderna, una versión gratuita muy completa, funciones sociales (seguir amigos y compartir entrenamientos) e integración con relojes inteligentes. Su versión premium ronda los 39 €/año.

Strong

Un diario de gimnasio popular y sencillo, valorado por su simplicidad. Permite registrar entrenamientos y destaca los récords personales, aunque su versión gratuita limita el número de rutinas guardadas. La versión Pro cuesta unos 4,99 €/mes.

JEFIT

Orientada a principiantes, su punto fuerte es una enorme biblioteca de ejercicios con demostraciones en vídeo.

FitNotes

Una opción totalmente gratuita para Android, muy funcional, pero con una interfaz anticuada y sin sincronización en la nube ni acceso multiplataforma.

En conjunto, el estado del arte muestra un mercado maduro donde las aplicaciones líderes ofrecen muchas funciones, pero casi siempre bajo un modelo de pago (suscripción premium) para desbloquear las características más avanzadas, como las estadísticas detalladas.

4.3. Innovación

ProgressFit no pretende competir directamente con gigantes como Hevy en número de funciones, sino ofrecer una alternativa **sencilla, gratuita y centrada en lo esencial**: registrar entrenamientos y, sobre todo, visualizar el progreso de forma clara.

Su aportación principal frente a la competencia es la integración, en una única herramienta gratuita, del registro de entrenamientos junto con el seguimiento del peso corporal y el cálculo automático de indicadores de salud (IMC y calorías diarias recomendadas según el objetivo), funciones que en otras aplicaciones suelen estar separadas o reservadas a las versiones de pago.

Como proyecto académico, su valor innovador reside también en ser una aplicación web full-stack completa y propia, desarrollada desde cero (frontend, backend y base de datos), que demuestra de forma práctica las tecnologías y competencias adquiridas durante el ciclo formativo.

4.4. DAFO

DAFO	Origen Interno	Origen externo
	Fortalezas	Oportunidades
Aspectos positivos	- Aplicación gratuita y sin anuncios. - Combina registro de entrenamientos, control de peso e indicadores de salud en un solo lugar. - Interfaz limpia y adaptada a móvil. - Control total sobre el código al ser desarrollo propio.	- Sector del fitness en crecimiento. - Tendencia a digitalizar y cuantificar la actividad física. - Existe público que busca alternativas gratuitas y sencillas frente a las apps de pago.
	Debilidades	Amenazas
Aspectos negativos	- Proyecto desarrollado por una sola persona y con tiempo limitado. - Menor número de funciones que las apps consolidadas. - Sin funciones sociales ni app móvil nativa. - Biblioteca de ejercicios inicial reducida.	- Competencia muy fuerte y consolidada (Hevy, Strong, JEFIT). - Apps competidoras con grandes equipos y recursos. - Usuarios ya fidelizados a otras aplicaciones.

5. Planificación

5.1. Temporalización del proyecto

El proyecto se inició en **octubre de 2025** y se ha desarrollado a lo largo de todo el curso académico 2025/2026.

Conviene explicar el ritmo de trabajo, ya que ha estado condicionado por dos factores. En primer lugar, a partir **de diciembre de 2025** compaginé el desarrollo del proyecto con un trabajo, por lo que avancé principalmente durante los fines de semana y los ratos disponibles entre clases. En segundo lugar, el proyecto vivió un **cambio tecnológico importante**: comenzó desarrollándose con HTML, CSS y JavaScript puro y, por recomendación del profesor, se decidió migrar el frontend a React.

Por este motivo, la primera mitad del curso se dedicó al aprendizaje de las tecnologías, al diseño del sistema, a una primera versión de la interfaz en HTML/CSS/JS y al desarrollo del backend, mientras que la segunda mitad se centró en la reconstrucción del frontend en React y en la integración final de todas las funcionalidades.

Esto explica que el historial de versiones de la versión definitiva se concentre en los meses finales, ya que corresponde a la reconstrucción del proyecto sobre la base ya diseñada y aprendida previamente.

PROGRESSFIT

Fase	Descripción	Periodo
1. Análisis y diseño	Definición de la idea, requisitos, diseño de la base de datos y primeros bocetos de la interfaz.	Octubre 2025
2. Aprendizaje y primera versión del frontend	Estudio de las tecnologías y desarrollo de una primera interfaz con HTML, CSS y JavaScript puro.	Octubre – Diciembre 2025
3. Desarrollo del backend	Configuración de Spring Boot y MySQL, modelo de datos, API REST con DTOs, validaciones y roles.	Diciembre 2025 – Febrero 2026
4. Autenticación y seguridad	Registro e inicio de sesión con JWT y Spring Security, condiciones de registro.	Febrero – Marzo 2026
5. Migración a React	Decisión de migrar y reconstrucción del frontend con React + Vite (login, registro, dashboard).	Marzo – Abril 2026
6. Entrenamientos y catálogo	Registro de entrenamientos, grupos musculares, ejercicios y series.	Abril – Mayo 2026
7. Estadísticas y progreso	Páginas de estadísticas con Recharts, seguimiento de peso, IMC y cálculo de calorías.	Mayo 2026
8. Perfil y registro por pasos	Perfil de usuario ampliado, registro guiado en varios pasos.	Mayo – Junio 2026
9. Panel de administración	Gestión de usuarios, músculos y ejercicios.	Junio 2026
10. Ajustes finales y despliegue	Recuperación de contraseña, contenedorización con Docker, correcciones y pruebas.	Junio 2026
11. Documentación	Redacción de la memoria del proyecto.	Junio 2026

5.2. Recursos humanos y materiales

Recursos humanos: el proyecto ha sido desarrollado íntegramente por **una sola persona**, el autor de esta memoria, que ha asumido todos los roles: análisis, diseño, programación del backend y del frontend, diseño de la base de datos, pruebas y documentación. Se ha contado además con la orientación del profesorado del ciclo.

Recursos materiales (hardware):

- Ordenador personal para el desarrollo.
- Dispositivo móvil para probar la aplicación en su entorno de uso real.

Recursos materiales (software): todas las herramientas utilizadas son gratuitas o de código abierto:

- IntelliJ IDEA Community Edition (desarrollo del backend).
- Visual Studio Code (desarrollo del frontend).
- MySQL y MySQL Workbench (base de datos).
- Git y GitHub (control de versiones).
- Navegador web (pruebas).

5.3. Presupuesto y financiación

Al tratarse de un proyecto académico desarrollado con herramientas gratuitas y de código abierto, el **coste real de software ha sido nulo**. No obstante, a efectos de estimar lo que costaría este proyecto en un entorno profesional, se presenta el siguiente presupuesto orientativo:

Concepto	Detalle	Coste estimado
Desarrollo (mano de obra)	Aprox. 300 horas de trabajo a un precio orientativo de 15 €/hora para un programador junior.	4.500 €
Software y herramientas	IDEs, base de datos y control de versiones (todo open source / gratuito).	0 €
Equipo informático	Ordenador de desarrollo (amortizado).	0 € (ya disponible)
Alojamiento (hosting) en producción	Servidor y base de datos en la nube (estimación anual).	~120 €/año
Dominio web	Registro de un dominio propio (estimación anual).	~12 €/año
Total desarrollo		4.500 €
Total mantenimiento anual		~132 €/año

En el marco de este proyecto académico, la financiación no ha sido necesaria, ya que no se ha incurrido en gastos reales. En un escenario real de puesta en producción, los costes de mantenimiento (hosting y dominio) podrían cubrirse mediante una versión premium de pago o publicidad.

6. Análisis del sistema

6.1. Definición de requisitos

Requisitos funcionales

Los requisitos funcionales describen las funcionalidades que el sistema debe ofrecer. Se indica entre paréntesis el objetivo específico que cubre cada uno.

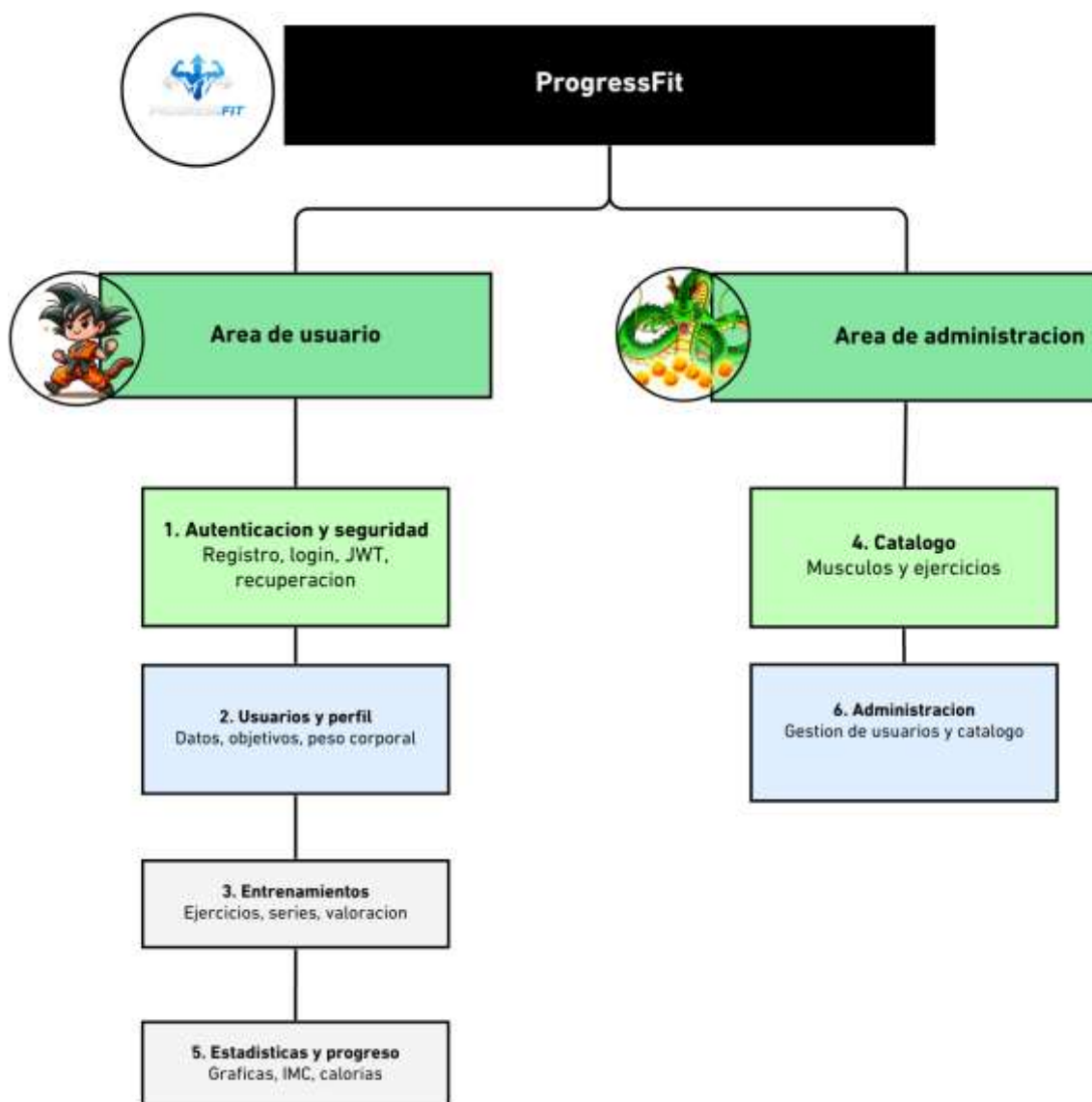
Código	Requisito funcional	Objetivo
RF-01	El sistema permitirá registrar nuevos usuarios mediante un formulario guiado en varios pasos.	Obj.1
RF-02	El sistema permitirá a los usuarios iniciar sesión mediante email y contraseña.	Obj.1
RF-03	El sistema validará que el email no esté ya registrado durante el registro.	Obj.1
RF-04	El sistema permitirá al usuario recuperar su contraseña verificando su identidad mediante email y fecha de nacimiento.	Obj.9
RF-05	El usuario podrá crear entrenamientos indicando nombre, ejercicios, series, repeticiones y peso.	Obj.2
RF-06	El usuario podrá consultar el detalle de un entrenamiento y editar o eliminar sus series.	Obj.2
RF-07	El usuario podrá registrar la valoración y la fatiga percibida al finalizar un entrenamiento.	Obj.2
RF-08	El sistema ofrecerá un catálogo de ejercicios organizados por grupos musculares.	Obj.3
RF-09	El usuario podrá registrar su peso corporal, con un máximo de un registro por día.	Obj.4
RF-10	El sistema calculará y mostrará el Índice de Masa Corporal (IMC) del usuario.	Obj.5
RF-11	El sistema estimará las calorías diarias recomendadas según el objetivo y nivel de actividad del usuario.	Obj.5
RF-12	El sistema mostrará gráficas de evolución del peso, fuerza, frecuencia y valoración de los entrenamientos.	Obj.6
RF-13	El usuario podrá consultar y modificar sus datos personales y objetivos desde su perfil.	Obj.7
RF-14	El administrador podrá gestionar los usuarios (cambiar rol y activar/desactivar).	Obj.8
RF-15	El administrador podrá crear y editar grupos musculares.	Obj.8
RF-16	El administrador podrá crear, editar, activar/desactivar y eliminar ejercicios.	Obj.8
RF-17	El sistema restringirá el acceso al panel de administración solo a usuarios con rol de administrador.	Obj.1,8

Requisitos no funcionales

Los requisitos no funcionales describen las características de calidad del sistema.

Código	Requisito no funcional
RNF-01	Seguridad: las contraseñas se almacenarán cifradas (BCrypt) y el acceso a los recursos protegidos se controlará mediante tokens JWT.
RNF-02	Usabilidad: la interfaz será intuitiva y sencilla, pensada para un uso rápido durante el entrenamiento.
RNF-03	Diseño adaptable (responsive): la interfaz se adaptará a dispositivos móviles, al ser el dispositivo de uso habitual.
RNF-04	Rendimiento: las consultas y respuestas del servidor deberán ser ágiles para no entorpecer el registro de entrenamientos.
RNF-05	Integridad de datos: se aplicará borrado lógico en usuarios y ejercicios para preservar el histórico y la integridad referencial.
RNF-06	Mantenibilidad: el código se estructurará de forma modular (separación de capas en el backend y de lógica/vista en el frontend) para facilitar su mantenimiento.
RNF-07	Portabilidad: la aplicación se podrá desplegar mediante contenedores Docker.

6.2. Identificación de los subsistemas



La aplicación se ha estructurado en varios subsistemas o módulos, cada uno responsable de un área concreta de funcionalidad. Esta división facilita el desarrollo, el mantenimiento y la comprensión del sistema.

1. Subsistema de autenticación y seguridad Gestiona el registro de nuevos usuarios, el inicio de sesión, la recuperación de contraseña y el control de acceso. Es el responsable de generar y validar los tokens JWT y de cifrar las contraseñas. Controla además qué partes de la aplicación puede ver cada usuario según su rol.

2. Subsistema de gestión de usuarios y perfil Permite a cada usuario consultar y modificar sus datos personales (nombre, altura, fecha de nacimiento, sexo), sus objetivos (peso objetivo y nivel de actividad) y su avatar. También gestiona el registro del peso corporal a lo largo del tiempo.

3. Subsistema de entrenamientos Es el núcleo de la aplicación. Permite al usuario crear y registrar sus entrenamientos, añadir ejercicios a cada entrenamiento, registrar las series (repeticiones, peso, RIR) y consultar el detalle de cada sesión. Incluye la valoración y la fatiga percibida de cada entrenamiento.

4. Subsistema de catálogo (músculos y ejercicios) Mantiene el catálogo de grupos musculares y los ejercicios asociados a cada uno. Este catálogo sirve de base para que el usuario construya sus entrenamientos.

5. Subsistema de estadísticas y progreso Procesa los datos registrados por el usuario y los convierte en información visual: gráficas de evolución del peso, de fuerza por ejercicio, de frecuencia semanal y de valoración. Calcula además indicadores como el IMC y la estimación de calorías diarias recomendadas.

6. Subsistema de administración Reservado a los usuarios con rol de administrador. Permite gestionar los usuarios de la aplicación (cambiar su rol y activarlos/desactivarlos), así como administrar el catálogo de músculos y ejercicios.

6.3. Análisis de casos de uso

Diagrama de caso de uso UML



CU-01: Registrar entrenamiento

Campo	Descripción
Identificador	CU-01
Nombre	Registrar un entrenamiento
Actor	Usuario
Objetivo relacionado	Obj. 2
Precondición	El usuario ha iniciado sesión.
Flujo principal	1. El usuario accede a la opción de crear un nuevo entrenamiento. 2. Introduce el nombre del entrenamiento. 3. Añade uno o varios ejercicios seleccionándolos del catálogo. 4. Para cada ejercicio, registra sus series (repeticiones, peso y RIR). 5. Al finalizar, indica la valoración y la fatiga percibida. 6. El sistema guarda el entrenamiento y lo asocia al usuario.
Flujo alternativo	Si el usuario no rellena los campos obligatorios, el sistema muestra un mensaje de error y no guarda.
Postcondición	El entrenamiento queda registrado y disponible en el historial del usuario.

CU-02: Registrar peso corporal

Campo	Descripción
Identificador	CU-02
Nombre	Registrar el peso corporal
Actor	Usuario
Objetivo relacionado	Obj. 4
Precondición	El usuario ha iniciado sesión.
Flujo principal	1. El usuario accede a su perfil. 2. Introduce su peso corporal actual. 3. El sistema comprueba si ya existe un registro para la fecha de hoy. 4. Si no existe, crea un nuevo registro; si existe, lo actualiza. 5. El sistema confirma que el peso se ha guardado.
Flujo alternativo	Si el peso introducido está fuera del rango válido (30-300 kg), el sistema muestra un error.
Postcondición	El peso queda registrado y se refleja en la evolución del usuario.

CU-03: Gestionar ejercicios (administrador)	
Campo	Descripción
Identificador	CU-03
Nombre	Gestionar ejercicios
Actor	Administrador
Objetivo relacionado	Obj. 8
Precondición	El usuario ha iniciado sesión y tiene rol de administrador.
Flujo principal	1. El administrador accede al panel de administración. 2. Selecciona la gestión de ejercicios. 3. Puede crear un ejercicio nuevo (nombre, músculo, dificultad, descripción, vídeo). 4. Puede editar, activar/desactivar o eliminar ejercicios existentes. 5. El sistema guarda los cambios.
Flujo alternativo	Si intenta eliminar un ejercicio que está siendo usado en entrenamientos, el sistema lo impide y le sugiere desactivarlo en su lugar.
Postcondición	El catálogo de ejercicios queda actualizado.

6.4. Análisis de interfaces

Antes de comenzar la programación de la interfaz, se realizaron unos bocetos de las pantallas principales para definir su estructura y distribución.

Estos bocetos son esquemáticos y se centran en la disposición de los elementos, no en el aspecto visual final. Dado que la aplicación está pensada principalmente para su uso en el móvil, los bocetos se diseñaron con formato de pantalla de teléfono.

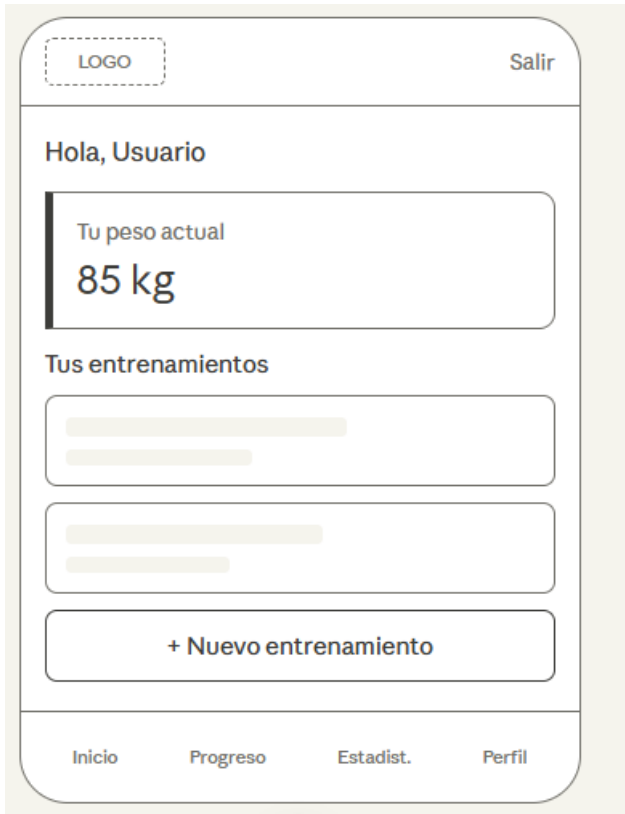
Se presentan los bocetos de las tres pantallas más representativas: la pantalla de **inicio de sesión**, el **panel principal** (dashboard) y la pantalla de **registro de un entrenamiento**, que constituye la función central de la aplicación. El resto de pantallas siguen los mismos patrones de diseño definidos en estos bocetos.

Login



The wireframe illustrates the layout of the login screen. At the top, there is a dashed rectangular box labeled "LOGO". Below it, the text "eslogan de la app" is centered. The form consists of three main input sections: an "Email" label followed by a wide text input field; a "Contraseña" label followed by a wide text input field; and a large rectangular button labeled "Entrar". At the bottom of the form, there are two lines of text: "¿Olvidaste tu contraseña?" and "¿No tienes cuenta? Regístrate", both in a smaller font size.

Dashboard



Dashboard UI mockup showing a user profile and workout history.

Header: LOGO (placeholder), Salir

Greeting: Hola, Usuario

Weight Section: Tu peso actual, 85 kg

Workouts Section: Tus entrenamientos

- Placeholder for workout details (yellow bar)
- Placeholder for workout details (yellow bar)
- + Nuevo entrenamiento

Footer: Inicio, Progreso, Estadist., Perfil

Registrar entrenamiento



New workout registration UI mockup.

Header: < Nuevo entrenamiento

Form Fields:

- Nombre (text input)
- Press banca (exercise name)
- Serie (text input)
- Reps (text input)
- Peso (text input)
- + Añadir serie (button)
- + Añadir ejercicio (button)
- Guardar entrenamiento (button)

6.5. Especificación del plan de pruebas

Para garantizar el correcto funcionamiento de la aplicación, se ha definido un plan de pruebas que cubre las funcionalidades principales del sistema. Se han considerado principalmente **pruebas funcionales** (comprobar que cada funcionalidad hace lo que debe), así como pruebas de **validación de datos**, de **seguridad/acceso** y de **usabilidad**.

Las pruebas se han realizado de forma manual, probando la aplicación como lo haría un usuario real y comprobando tanto los casos correctos como los casos de error.

Pruebas funcionales			
ID	Prueba	Resultado esperado	Requisito
PF-01	Registrar un usuario con datos válidos.	El usuario se crea y puede iniciar sesión.	RF-01
PF-02	Registrar un usuario con un email ya existente.	El sistema muestra un error y no crea el usuario.	RF-03
PF-03	Iniciar sesión con credenciales correctas.	El usuario accede a la aplicación.	RF-02
PF-04	Iniciar sesión con contraseña incorrecta.	El sistema muestra "Email o contraseña incorrectos".	RF-02
PF-05	Recuperar contraseña con email y fecha de nacimiento correctos.	El sistema permite cambiar la contraseña.	RF-04
PF-06	Recuperar contraseña con fecha de nacimiento incorrecta.	El sistema muestra "Los datos no coinciden".	RF-04
PF-07	Crear un entrenamiento con ejercicios y series.	El entrenamiento se guarda y aparece en el historial.	RF-05
PF-08	Registrar el peso corporal.	El peso se guarda y se refleja en el progreso.	RF-09
PF-09	Registrar el peso dos veces el mismo día.	El sistema actualiza el registro en lugar de duplicarlo.	RF-09
PF-10	Consultar la página de progreso.	Se muestran el IMC, las calorías y las gráficas correctamente.	RF-10, RF-11, RF-12
PF-11	Crear un ejercicio desde el panel de administración.	El ejercicio se crea y aparece en el catálogo.	RF-16
PF-12	Eliminar un ejercicio que está en uso en un entrenamiento.	El sistema lo impide y sugiere desactivarlo.	RF-16

Pruebas de validación de datos

ID	Prueba	Resultado esperado
PF-01	Introducir una contraseña de menos de 6 caracteres.	El sistema muestra un error de validación.
PV-02	Introducir un peso fuera del rango (30-300 kg).	El sistema rechaza el valor.
PV-03	Dejar campos obligatorios vacíos en un formulario.	El sistema no permite continuar y avisa al usuario.
PV-04	Introducir una fecha de nacimiento futura.	El sistema rechaza la fecha.

Pruebas de seguridad y control de acceso

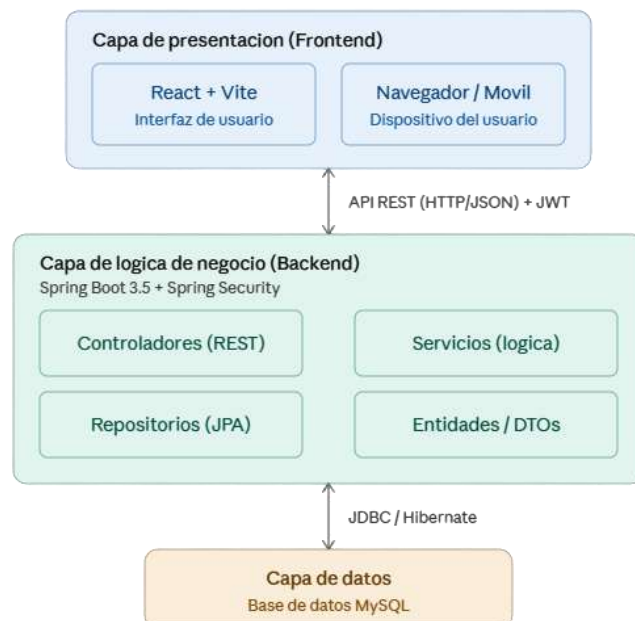
ID	Prueba	Resultado esperado
PS-01	Acceder a una ruta protegida sin haber iniciado sesión.	El sistema redirige al inicio de sesión.
PS-02	Acceder al panel de administración con un usuario normal.	El sistema deniega el acceso y redirige.
PS-03	Comprobar que las contraseñas se guardan cifradas en la base de datos.	Las contraseñas aparecen cifradas (BCrypt), nunca en texto plano.
PS-04	Iniciar sesión con un usuario desactivado.	El sistema impide el acceso.

Pruebas de usabilidad

ID	Prueba	Resultado esperado
PU-01	Navegar por la aplicación desde un teléfono móvil.	La interfaz se adapta correctamente (responsive).
PU-02	Comprobar que los mensajes de error son claros y comprensibles.	El usuario entiende qué ha fallado.
PU-03	Comprobar que las acciones principales son accesibles en pocos toques.	El registro de un entrenamiento es rápido e intuitivo.

7. Diseño del sistema

7.1 Arquitectura del sistema



ProgressFit sigue una **arquitectura cliente-servidor organizada en tres capas**, un patrón ampliamente utilizado en el desarrollo de aplicaciones web por su claridad y mantenibilidad. Cada capa tiene una responsabilidad bien definida y se comunica con las demás a través de interfaces concretas.

Capa de presentación (Frontend). Es la parte con la que interactúa el usuario. Está desarrollada con **React** y la herramienta de construcción **Vite**, y se ejecuta en el navegador del dispositivo del usuario (ordenador o móvil). Se encarga de mostrar la interfaz y de recoger las acciones del usuario, pero no contiene lógica de negocio: cada vez que necesita datos, los solicita al backend.

Capa de lógica de negocio (Backend). Es el servidor, desarrollado con **Spring Boot** y protegido con **Spring Security**. Recibe las peticiones del frontend, aplica la lógica de negocio (validaciones, cálculos, reglas) y gestiona la seguridad. Internamente se organiza, a su vez, en varias subcapas siguiendo el patrón habitual de Spring:

- **Controladores:** reciben las peticiones REST y devuelven las respuestas.
- **Servicios:** contienen la lógica de negocio.
- **Repositorios:** acceden a la base de datos mediante JPA.

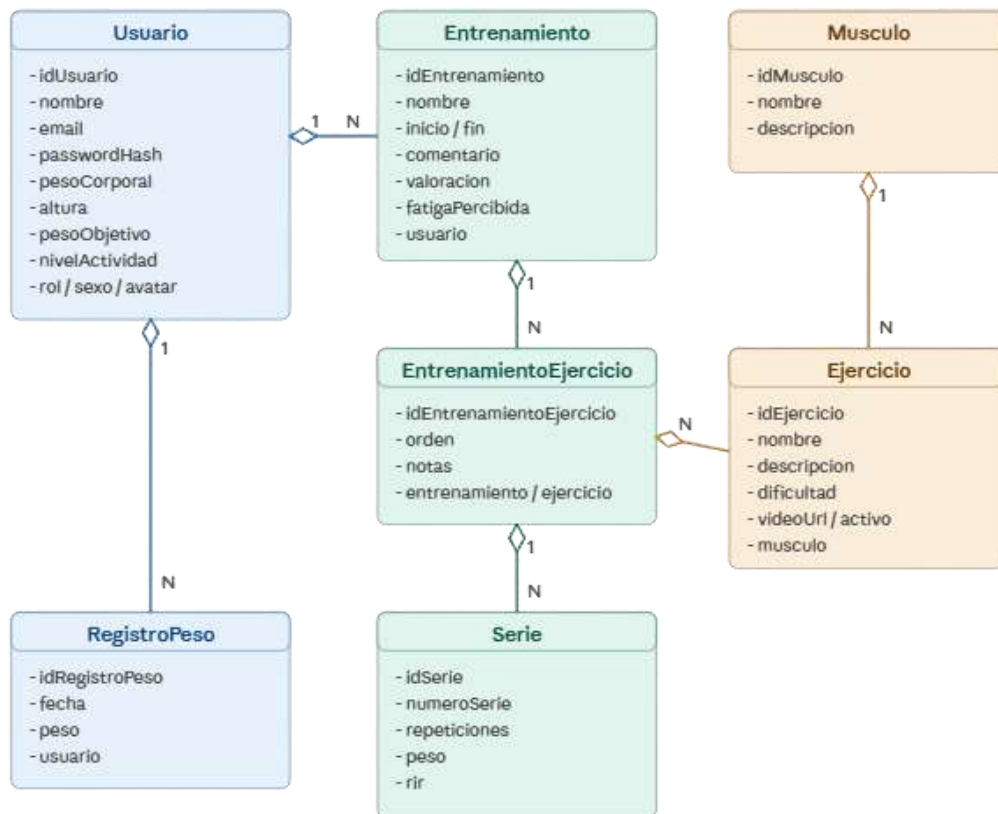
- **Entidades y DTOs:** las entidades representan las tablas, y los DTOs (*Data Transfer Objects*) se usan para intercambiar datos con el frontend sin exponer directamente las entidades.

Capa de datos. Es la base de datos **MySQL**, donde se almacena toda la información de forma persistente. El backend se comunica con ella a través de JDBC y Hibernate (la implementación de JPA).

Comunicación entre capas. El frontend y el backend se comunican mediante una **API REST**, intercambiando datos en formato JSON sobre el protocolo HTTP. La seguridad se gestiona mediante **tokens JWT**: tras iniciar sesión, el usuario recibe un token que debe enviar en cada petición a recursos protegidos, lo que permite al servidor verificar su identidad y permisos sin necesidad de mantener una sesión en memoria (arquitectura *stateless*). Por su parte, el backend se comunica con la base de datos mediante JDBC/Hibernate.

Esta separación en capas aporta varias ventajas: cada parte puede desarrollarse y mantenerse de forma independiente, el código queda más organizado, y se podría sustituir una capa (por ejemplo, cambiar la base de datos o rehacer el frontend) sin afectar a las demás. De hecho, durante el desarrollo del proyecto el frontend se rehízo por completo (migración de HTML/JS a React) sin necesidad de modificar el backend, lo que demuestra en la práctica la utilidad de esta arquitectura.

7.2.Diseño de clases



El backend está desarrollado siguiendo el paradigma de **programación orientada a objetos**. Las clases principales del sistema son las **entidades**, que representan los objetos del dominio del problema y se corresponden con las tablas de la base de datos. Cada entidad se mapea a su tabla mediante anotaciones de JPA (*Java Persistence API*).

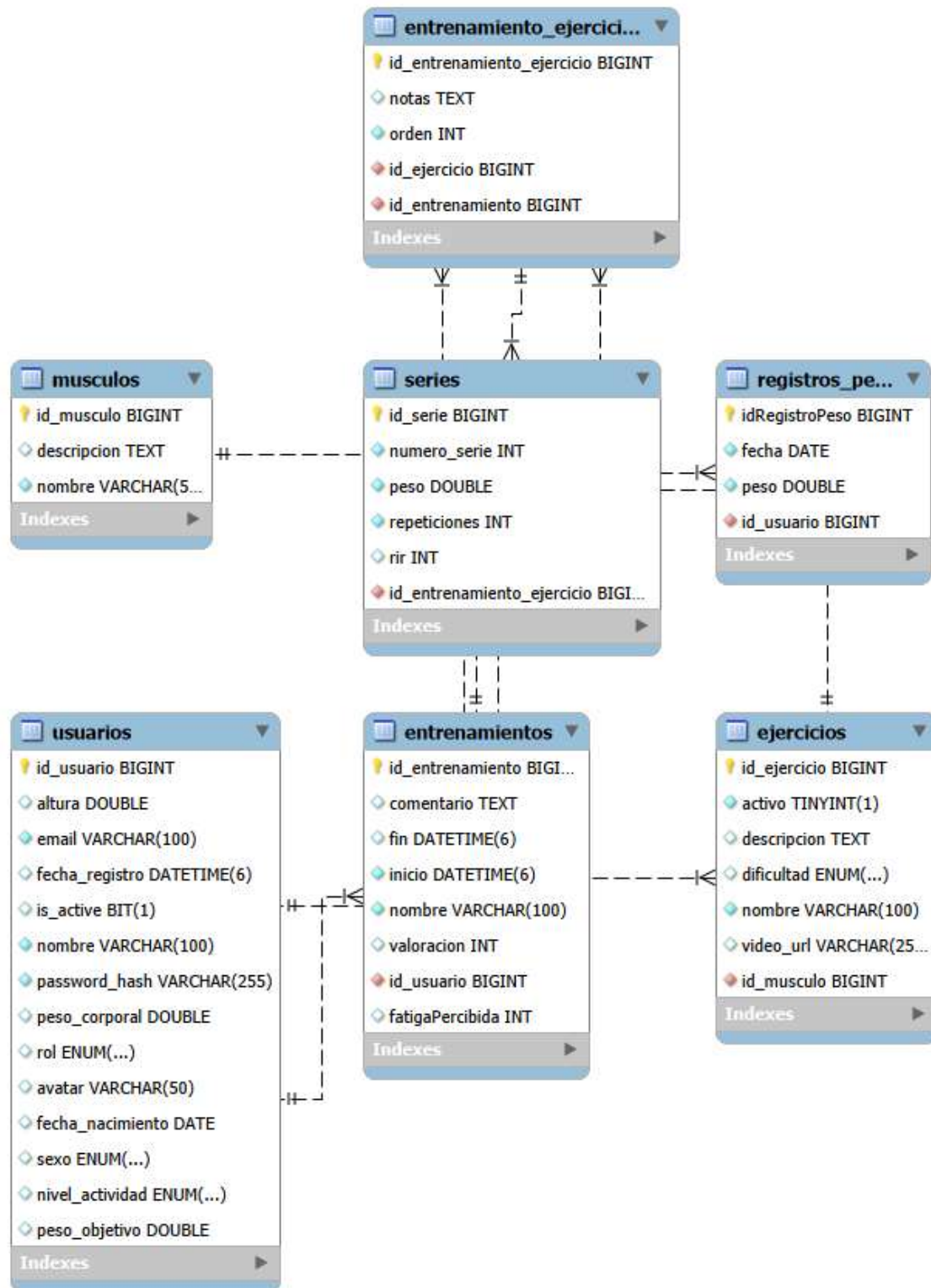
Las clases entidad principales son:

- **Usuario:** representa a un usuario de la aplicación, con sus datos personales, físicos y de cuenta.
- **Entrenamiento:** una sesión de entrenamiento, perteneciente a un usuario.
- **EntrenamientoEjercicio:** clase intermedia que relaciona un entrenamiento con un ejercicio concreto, permitiendo la relación muchos a muchos entre ambos.
- **Serie:** una serie concreta (repeticiones, peso y RIR) de un ejercicio dentro de un entrenamiento.
- **Ejercicio:** un ejercicio del catálogo, asociado a un grupo muscular.
- **Musculo:** un grupo muscular del catálogo.
- **RegistroPeso:** un registro del peso corporal del usuario en una fecha concreta.

Las relaciones entre las clases reflejan las del modelo de datos: un Usuario tiene muchos Entrenamiento y muchos RegistroPeso; un Entrenamiento se compone de varios EntrenamientoEjercicio, y cada uno de estos contiene varias Serie; un Musculo agrupa muchos Ejercicio. Estas relaciones se implementan en Java mediante anotaciones de JPA como @OneToMany y @ManyToOne.

Además de las entidades, el backend utiliza otras clases organizadas por su responsabilidad, siguiendo el patrón de capas de Spring: las clases **Controlador** (que exponen los *endpoints* REST), las clases **Servicio** (que contienen la lógica de negocio), las interfaces **Repositorio** (que gestionan el acceso a datos) y los **DTOs** (objetos para transferir datos entre el backend y el frontend sin exponer las entidades directamente).

7.3. Diseño de la base de datos



La base de datos de ProgressFit se ha implementado en MySQL y está formada por **siete tablas** relacionadas entre sí.

El diseño sigue un modelo relacional normalizado que evita la duplicación de datos y mantiene la integridad mediante claves foráneas. A continuación, se describe cada tabla y, posteriormente, las relaciones entre ellas. El diagrama Entidad-Relación se ha generado mediante ingeniería inversa desde la propia base de datos, por lo que refleja fielmente la implementación real.

Tablas

usuarios — Almacena los datos de cada usuario registrado: identificador (`id_usuario`, clave primaria), nombre, email (único), contraseña cifrada (`password_hash`), datos físicos y objetivos (`peso_corporal`, `altura`, `peso_objetivo`, `nivel_actividad`), datos personales (`fecha_nacimiento`, `sexo`), fecha de alta y estado (`fecha_registro`, `is_active`), rol del usuario (`rol`) y avatar.

entrenamientos — Cada sesión de entrenamiento registrada por un usuario: identificador (`id_entrenamiento`), nombre, fechas de inicio y fin, comentario, valoración y fatiga percibida. Incluye la clave foránea `id_usuario`, que indica a qué usuario pertenece.

musculos — Catálogo de grupos musculares: identificador (`id_musculo`), nombre y descripción.

ejercicios — Catálogo de ejercicios: identificador (`id_ejercicio`), nombre, descripción, dificultad, URL de vídeo y estado (activo, para el borrado lógico). Incluye la clave foránea `id_musculo`, que indica el grupo muscular al que pertenece.

entrenamiento_ejercicios — Tabla intermedia que relaciona los entrenamientos con los ejercicios que contienen, resolviendo una relación muchos a muchos. Almacena el orden del ejercicio dentro del entrenamiento, notas opcionales, y las claves foráneas `id_entrenamiento` e `id_ejercicio`.

series — Las series concretas realizadas de cada ejercicio dentro de un entrenamiento: identificador (id_serie), número de serie, repeticiones, peso y RIR. Incluye la clave foránea id_entrenamiento_ejercicio.

registros_peso — El histórico del peso corporal del usuario: identificador (idRegistroPeso), fecha y peso. Incluye la clave foránea id_usuario.

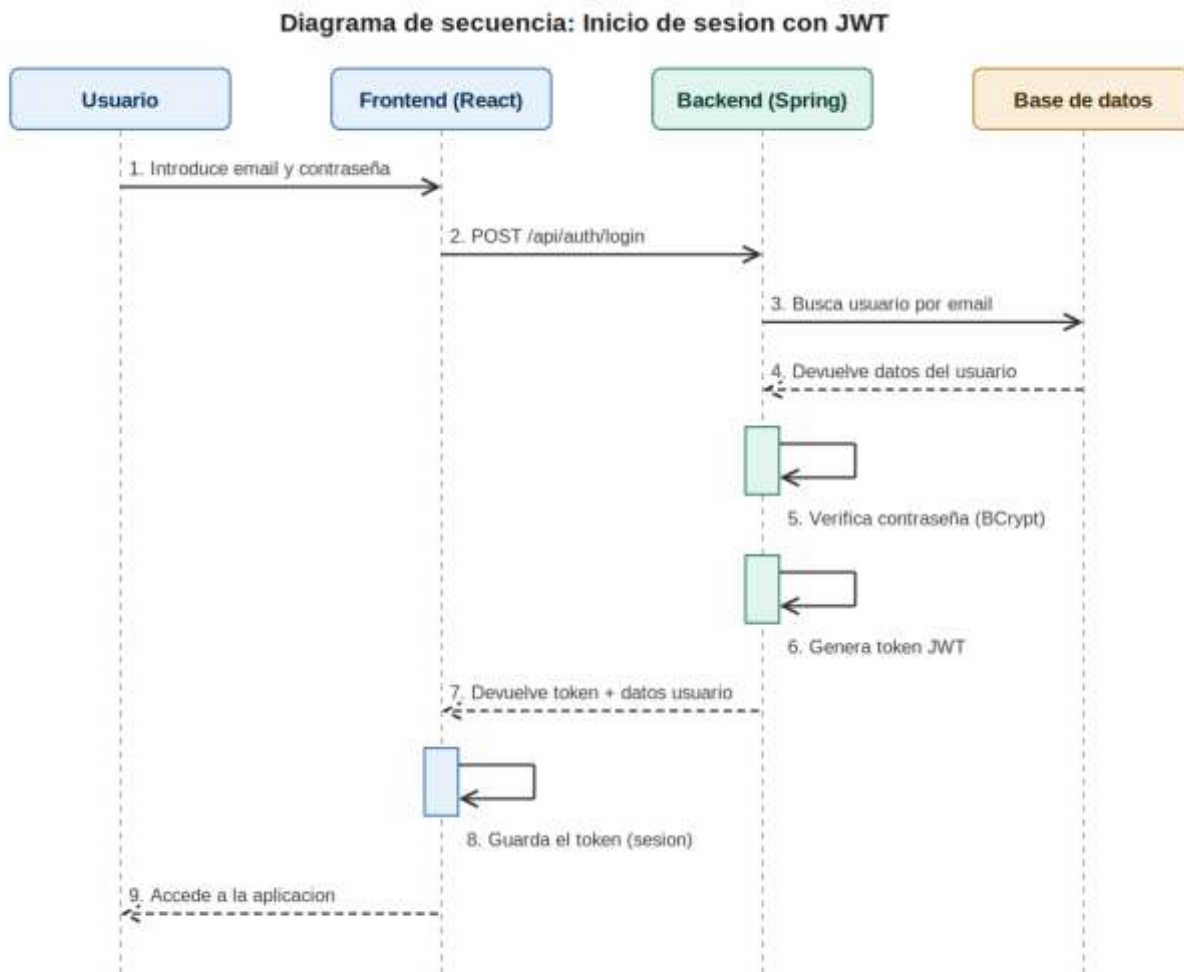
Relaciones

Las relaciones entre las tablas son las siguientes:

- Un **usuario** puede tener muchos **entrenamientos** (1:N), pero cada entrenamiento pertenece a un solo usuario.
- Un **usuario** puede tener muchos **registros de peso** (1:N).
- Un **músculo** puede tener muchos **ejercicios** (1:N), pero cada ejercicio pertenece a un solo músculo.
- Un **entrenamiento** y un **ejercicio** se relacionan a través de la tabla intermedia **entrenamiento_ejercicios**, que representa una relación de muchos a muchos (N:M).
- Cada fila de **entrenamiento_ejercicios** puede tener varias **series** (1:N), que son las series concretas realizadas de ese ejercicio en ese entrenamiento.

Este diseño permite que un usuario registre un entrenamiento con varios ejercicios, y que cada ejercicio tenga sus propias series con su peso y repeticiones, manteniendo toda la información relacionada y sin duplicar datos.

7.4 Otros diagramas



Para ilustrar el funcionamiento de uno de los procesos más importantes del sistema, se incluye un **diagrama de secuencia del inicio de sesión con autenticación JWT**. Este diagrama muestra cómo interactúan las distintas partes de la aplicación (usuario, frontend, backend y base de datos) a lo largo del tiempo durante el proceso de autenticación.

El flujo es el siguiente:

1. El usuario introduce su email y contraseña en el formulario de inicio de sesión.
2. El frontend envía estas credenciales al backend mediante una petición POST a la ruta /api/auth/login.
3. El backend busca en la base de datos un usuario con ese email.
4. La base de datos devuelve los datos del usuario, incluida su contraseña cifrada.

5. El backend verifica que la contraseña introducida coincide con la almacenada, comparándolas mediante el algoritmo BCrypt.
6. Si la contraseña es correcta, el backend genera un **token JWT** que contiene la identidad y el rol del usuario.
7. El backend devuelve al frontend el token junto con los datos del usuario.
8. El frontend guarda el token, que se usará en las siguientes peticiones para acceder a los recursos protegidos.
9. El usuario accede a la aplicación.

A partir de ese momento, en cada petición a una ruta protegida, el frontend envía el token JWT, y el backend lo válida para comprobar la identidad y los permisos del usuario sin necesidad de mantener una sesión en memoria (modelo *stateless*).

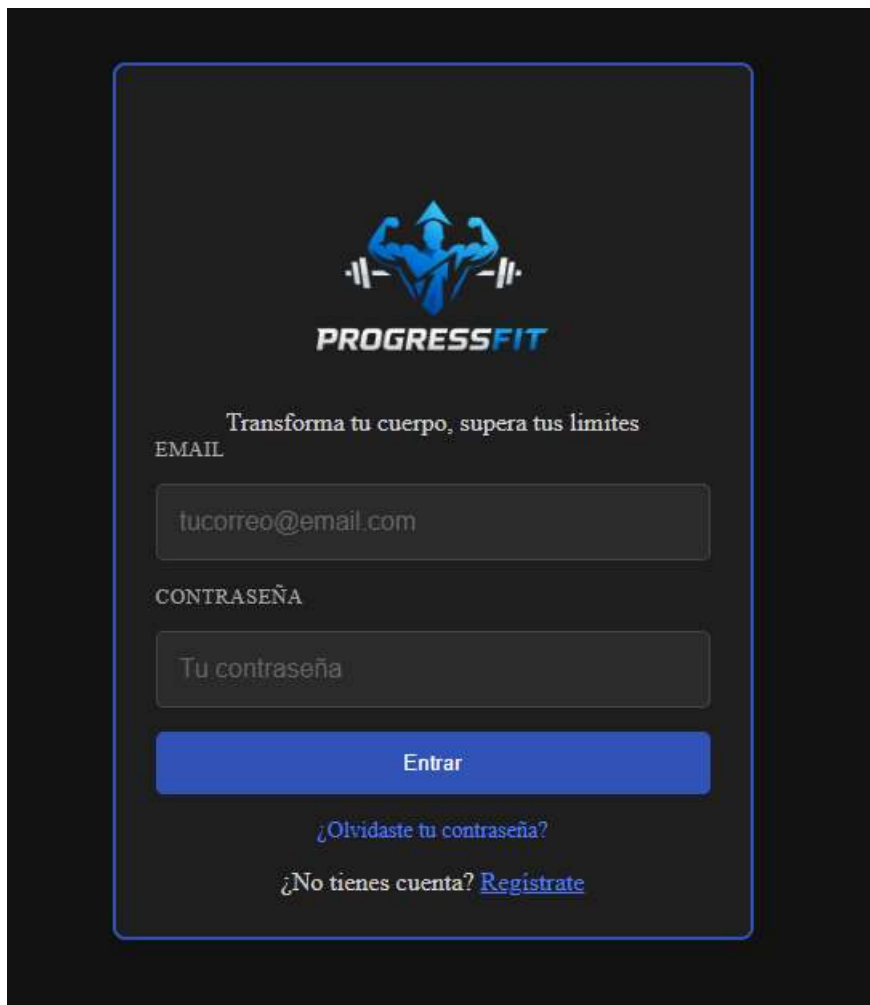
7.5 Diseño de interfaces

Una vez implementada la aplicación, se presentan las interfaces finales mediante capturas de pantalla reales. A diferencia de los bocetos mostrados en el apartado de análisis, estas capturas reflejan el aspecto definitivo de la aplicación, con su diseño visual completo: la paleta de colores oscura con acentos en azul, la tipografía y la disposición final de los elementos.

El diseño visual se ha mantenido coherente en todas las pantallas, utilizando una paleta de tonos oscuros (fondos en negro y gris oscuro) con un color de acento azul, buscando un aspecto moderno y un buen contraste que facilite la lectura, especialmente en el uso desde el móvil. La interfaz es totalmente adaptable (*responsive*), ajustándose tanto a pantallas de ordenador como de teléfono.

A continuación, se muestran las pantallas principales de la aplicación:

1. Pantalla de inicio de sesión




PROGRESSFIT

Transforma tu cuerpo, supera tus limites

EMAIL

tucorreo@email.com

CONTRASEÑA

Tu contraseña

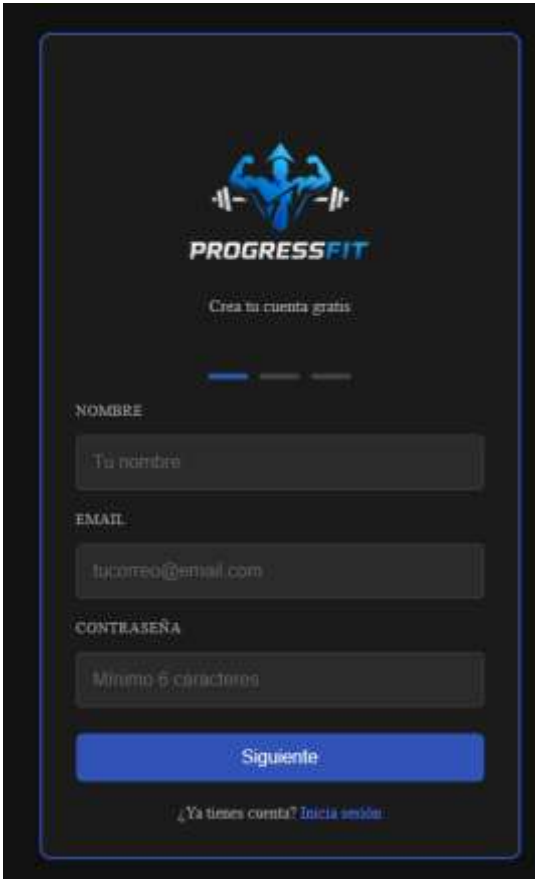
Entrar

[¿Olvidaste tu contraseña?](#)

[¿No tienes cuenta? Regístrate](#)

Permite al usuario acceder con su email y contraseña, con enlaces al registro y a la recuperación de contraseña.

2. Registro



PROGRESSFIT

Crea tu cuenta gratis

NOMBRE

Tu nombre

EMAIL

tucorreo@email.com

CONTRASEÑA

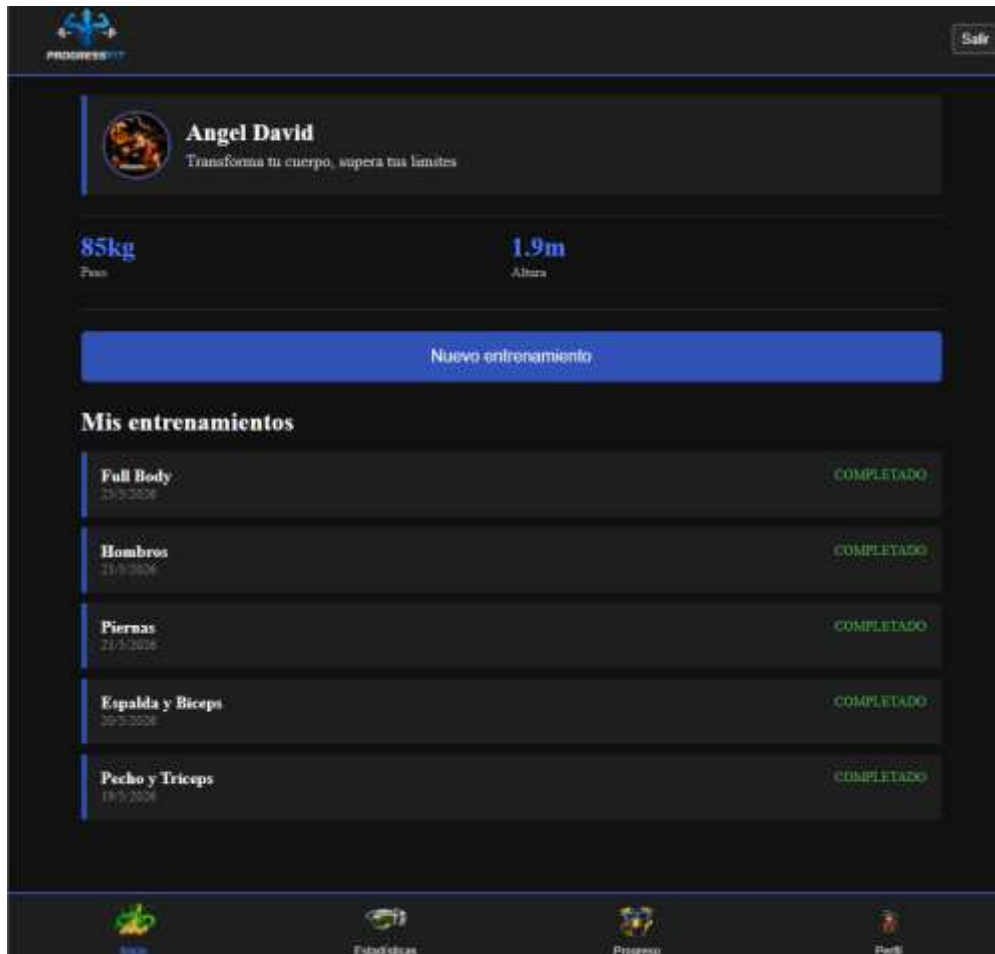
Mínimo 6 caracteres

Siguiente

¿Ya tienes cuenta? [Inicia sesión](#)

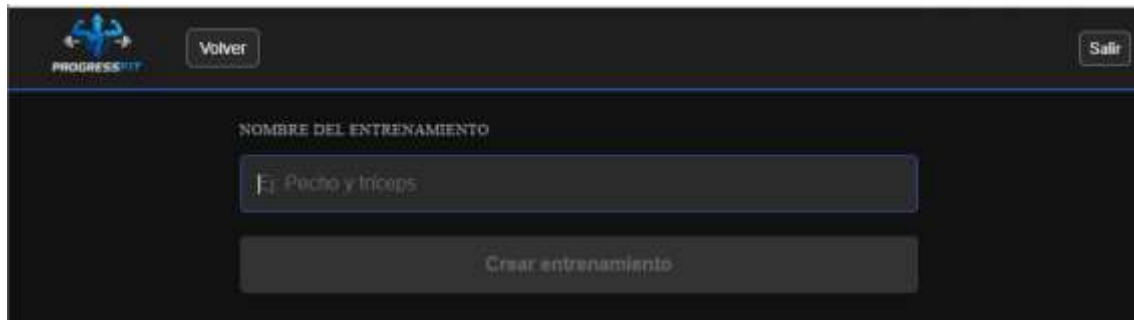
El registro se divide en varios pasos (datos de cuenta, datos personales y medidas) para no abrumar al usuario con un formulario largo.

3. Dashboard / panel principal



Muestra un resumen del usuario y el acceso a sus entrenamientos, con la barra de navegación inferior.

4. Crear/registrar entrenamiento



PROGRESSFIT

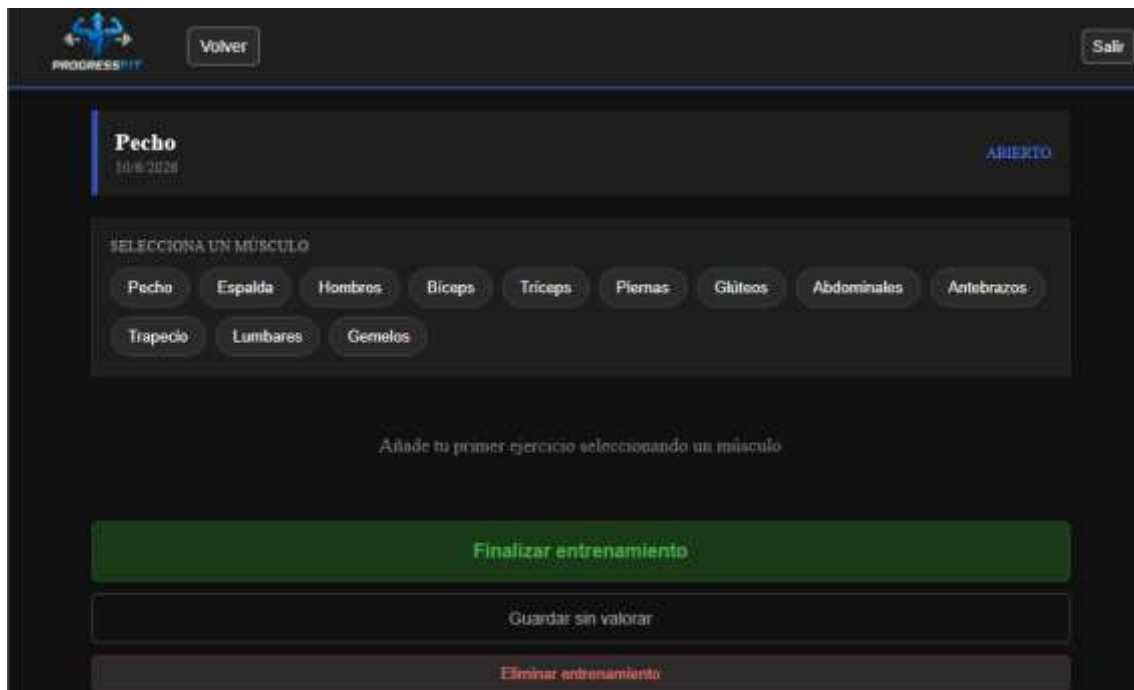
Volver

Salir

NOMBRE DEL ENTRENAMIENTO

Ej. Pecho y tríceps

Crear entrenamiento



PROGRESSFIT

Volver

Salir

Pecho
10/6/2026

ABIERTO

SELECCIONA UN MÚSCULO

Pecho Espalda Hombros Biceps Tríceps Piernas Glúteos Abdominales Antebrazos

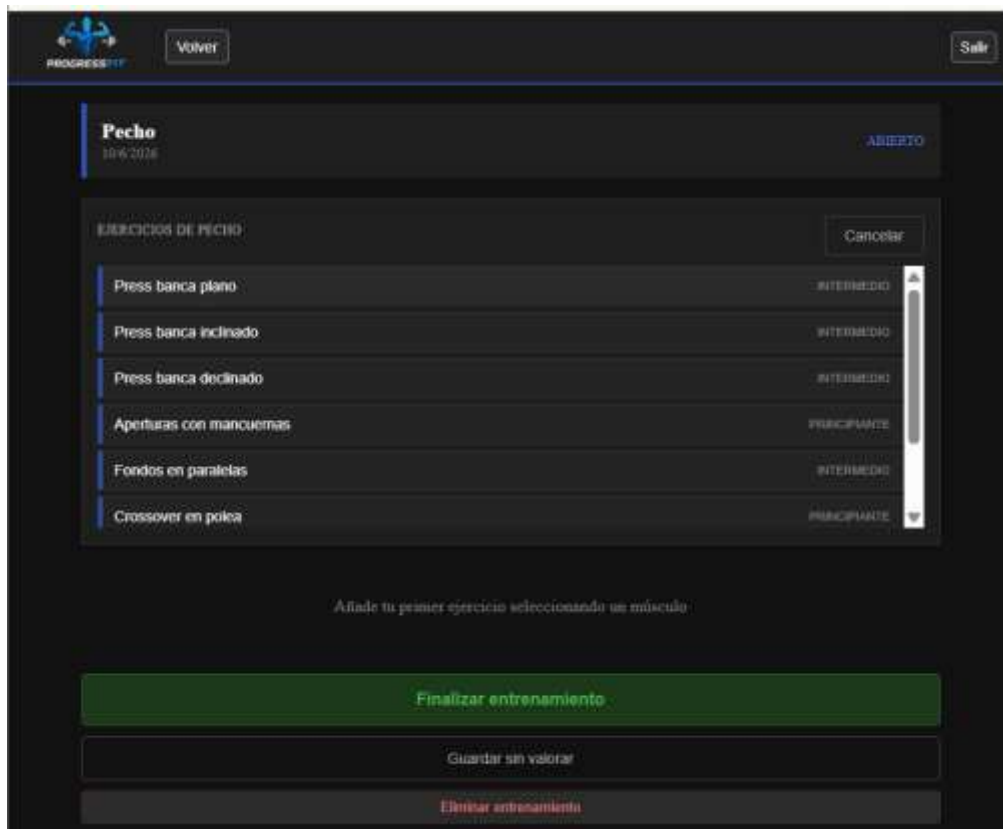
Trapecio Lumbares Gemelos

Añade tu primer ejercicio seleccionando un músculo

Finalizar entrenamiento

Guardar sin valorar

Eliminar entrenamiento



Pecho 10/6/2024 ABIERTO

EJERCICIOS DE PECHO Cancelar

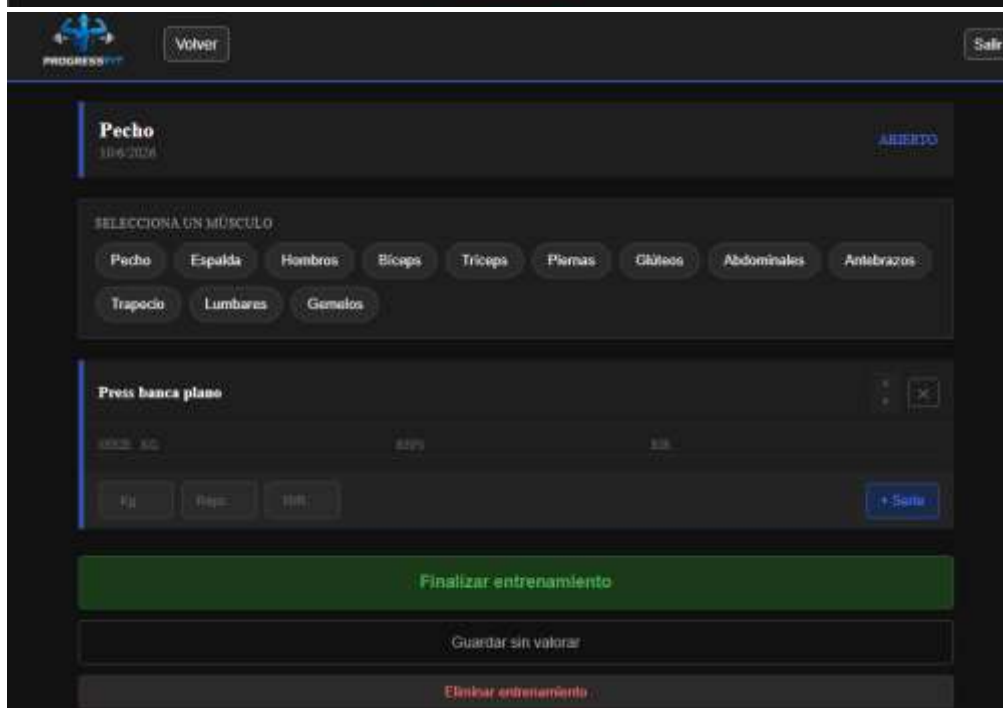
Press banca plano	INTERMEDIO
Press banca inclinado	INTERMEDIO
Press banca declinado	INTERMEDIO
Aperturas con mancuernas	PRINCIPIANTE
Fondos en paralelas	INTERMEDIO
Crossover en polea	PRINCIPIANTE

Añade tu primer ejercicio seleccionando un músculo

Finalizar entrenamiento

Guardar sin valorar

Eliminar entrenamiento



Pecho 10/6/2024 ABIERTO

SELECCIONA UN MÚSCULO

Pecho Espalda Hombros Bíceps Tríceps Piernas Glúteos Abdominales Antebrazos

Trapecio Lumbares Gemelos

Press banca plano + ×

SETS: 3 REPS: 10 PESO: 50kg

1 2 3 + Set

Finalizar entrenamiento

Guardar sin valorar

Eliminar entrenamiento

El usuario añade ejercicios y registra las series con sus repeticiones y peso.

5. Detalle de un entrenamiento

Full Body 25/5/2026 COMPLETADO

Press banca plano

SERIE	KG	REPES	RPE
S1	65kg	12	3
S2	72kg	10	2
S3	77kg	8	1

Dominadas

SERIE	KG	REPES	RPE
S1	0kg	9	3
S2	0kg	8	2
S3	0kg	7	1

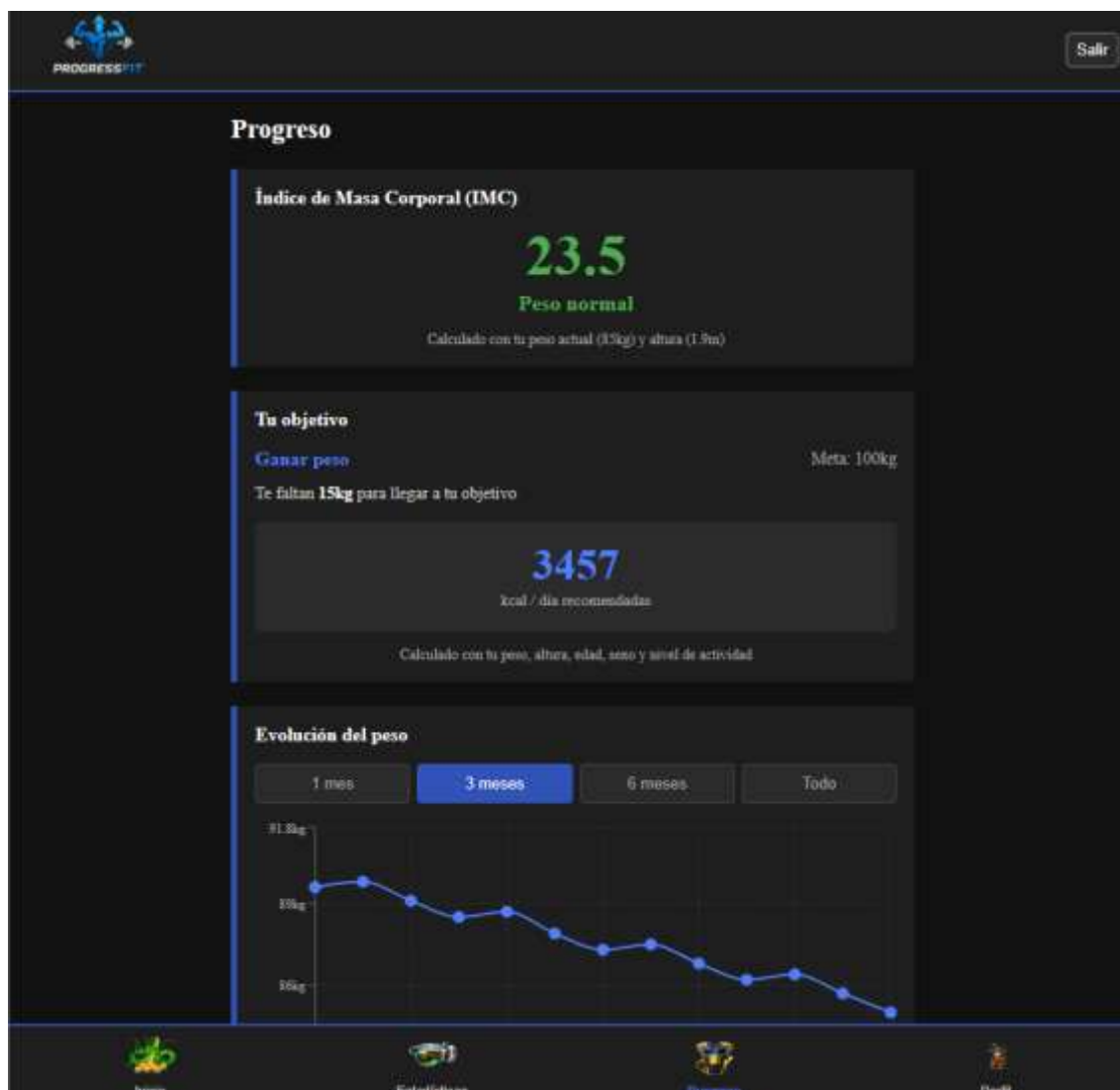
Press militar con barra

SERIE	KG	REPES	RPE
S1	45kg	10	3
S2	52kg	8	2
S3	57kg	6	1

[Editar valoración](#)

Permite consultar y editar las series de cada ejercicio.

6. Página de progreso (IMC, calorías, objetivo)





Muestra el IMC, la estimación de calorías diarias, el objetivo de peso y la evolución del peso corporal.

7. Página de estadísticas (gráficas)





Presenta varias gráficas con la evolución del peso, la fuerza, la frecuencia semanal y la valoración de los entrenamientos.

8. Perfil de usuario


Volver
Salir



Angel David
angelDavid837@gmail.com

Cambiar avatar

Datos
Peso
Objetivo

Datos personales
NOMBRE

ALTURA (M)

FECHA DE NACIMIENTO

SEXO

NUEVA CONTRASEÑA

Guardar cambios







Volver
Salir



Angel David
angeladavid857@gmail.com

Cambiar avatar

Datos
Peso
Objetivo

Peso corporal

Actualiza tu peso para llevar un registro de tu evolución.

PESO ACTUAL (KG)

85

Actualizar peso

Inicio
Estadísticas
Progreso
Perfil

Angel David
angel david857@gmail.com

Volver

Salir

Cambiar avatar

Datos

Peso

Objetivo

Mi objetivo
Define tu meta para calcular tus calorías recomendadas

PESO OBJETIVO (KG)

100

NIVEL DE ACTIVIDAD

Moderado (3-5 días/semana)

Guardar objetivo

Inicio

Estadísticas

Progreso

Perfil

Permite consultar y modificar los datos personales, los objetivos y registrar el peso corporal.

9. Panel de administración (menú)

Usuarios
Gestionar cuentas y roles

Músculos
Crear y editar músculos

Ejercicios
Crear y editar ejercicios

Cerrar sesión

Acceso reservado a administradores, con la gestión de usuarios, músculos y ejercicios.

10. Alguna gestión del admin (ej. ejercicios)

Cerrar sesión

Volver al menú

Gestión de ejercicios

Nuevo ejercicio

Buscar ejercicio...

10 ▾

Press banca plano Fecha: INTERMEDIO	Activo Editar	Eliminar
Press banca inclinado Fecha: INTERMEDIO	Activo Editar	Eliminar
Press banca declinado Fecha: INTERMEDIO	Activo Editar	Eliminar
Aperturas con mancuernas	Activo Editar	

Cerrar sesión

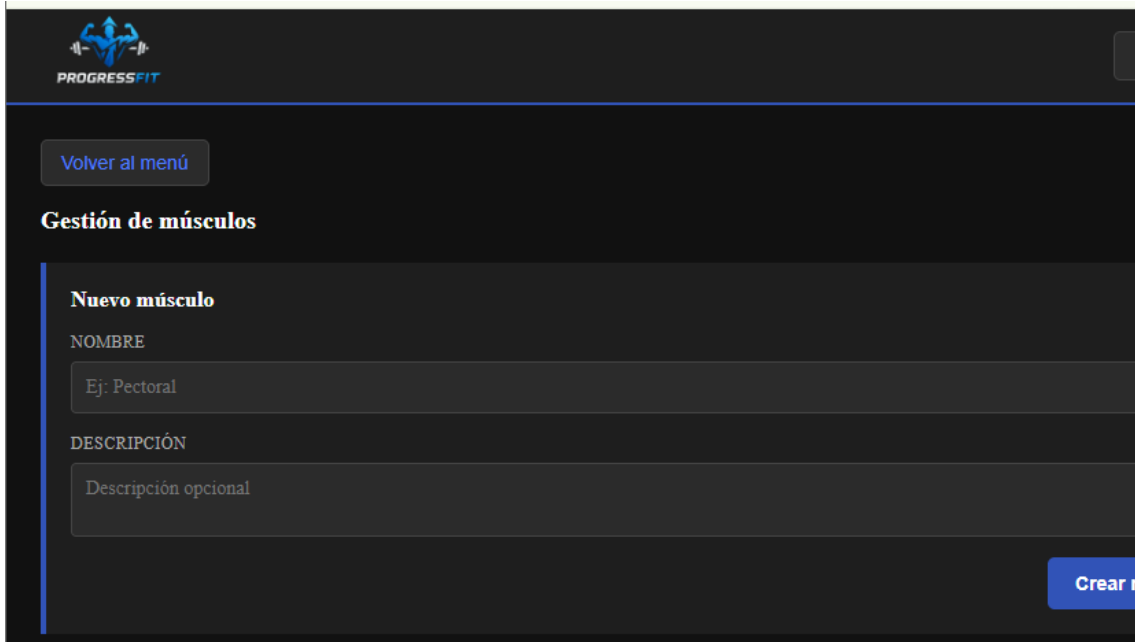
Volver al menú

Gestión de usuarios

Buscar por nombre o email...

10 ▾

USUARIO	ROL	ESTADO
admin admin@hotmail.es	Admin ▾	Activo
Usuario user@hotmail.es	Usuario ▾	Activo
Angel David angelidavid117@gmail.com	Usuario ▾	Activo
test12 test12@hotmail.es	Usuario ▾	Activo



Volver al menú

Gestión de músculos

Nuevo músculo

NOMBRE

Ej: Pectoral

DESCRIPCIÓN

Descripción opcional

Crear n

Gestión de ejercicios desde el panel de administración.

8.Implementación

8.1. Lenguajes, frameworks y tecnologías

8.1. Lenguajes, frameworks y tecnologías

Para el desarrollo de ProgressFit se han utilizado las siguientes tecnologías, organizadas por capas:

Backend

- **Java 17:** lenguaje de programación del lado del servidor.
- **Spring Boot 3.5:** framework principal del backend, que facilita la creación de aplicaciones web y APIs REST.
- **Spring Security:** para la gestión de la autenticación y autorización.
- **JWT (JSON Web Token):** para la autenticación sin estado (*stateless*) mediante tokens.
- **Spring Data JPA / Hibernate:** para el mapeo objeto-relacional y el acceso a la base de datos.
- **Maven:** herramienta de gestión de dependencias y construcción del proyecto.

- **Lombok:** para reducir el código repetitivo en las clases (getters, setters, constructores).

Frontend

- **React:** librería para construir la interfaz de usuario mediante componentes.
- **Vite:** herramienta de construcción y servidor de desarrollo del frontend.
- **JavaScript (JSX):** lenguaje de programación del cliente.
- **React Router:** para la navegación entre páginas.
- **Axios:** para realizar las peticiones HTTP al backend.
- **Recharts:** librería para la creación de las gráficas de estadísticas.
- **CSS:** para los estilos, organizados por componentes.

Base de datos

- **MySQL 8:** sistema gestor de base de datos relacional.

Herramientas de desarrollo

- **IntelliJ IDEA Community Edition:** entorno de desarrollo del backend.
- **Visual Studio Code:** editor para el frontend.
- **MySQL Workbench:** gestión y diseño de la base de datos.
- **Git y GitHub:** control de versiones y alojamiento del repositorio.
- **Docker:** para la contenedorización y el despliegue de la aplicación.
- **Postman / navegador:** para probar los *endpoints* de la API.

La arquitectura del backend sigue el patrón de capas habitual de Spring (controladores, servicios, repositorios y entidades), y se emplean **DTOs** para el intercambio de datos con el frontend. En el frontend se ha seguido un patrón de separación entre la lógica y la presentación: cada página se compone de un componente de vista (que solo se encarga de mostrar la interfaz) y un *hook* personalizado (que contiene toda la lógica y el estado), lo que facilita el mantenimiento y la legibilidad del código.

8.2. Problemas encontrados y soluciones

Durante el desarrollo del proyecto surgieron varios problemas que requirieron tomar decisiones técnicas. Los más relevantes fueron los siguientes:

Migración de la tecnología del frontend

El proyecto comenzó desarrollándose con HTML, CSS y JavaScript puro. A medida que crecía, el código se volvía difícil de mantener y de organizar. Por recomendación del profesor, se decidió migrar el frontend a **React**. Aunque supuso rehacer toda la interfaz, la migración aportó una estructura basada en componentes mucho más ordenada y mantenible. Gracias a la arquitectura en capas, el backend no tuvo que modificarse, ya que la comunicación seguía siendo la misma API REST.

Almacenamiento seguro de las contraseñas

Por motivos de seguridad, las contraseñas no podían guardarse en texto plano. Se resolvió utilizando **BCrypt** (a través de Spring Security), que cifra las contraseñas antes de almacenarlas. De este modo, ni siquiera desde la base de datos es posible conocer la contraseña original de un usuario.

Borrado de datos con relaciones (claves foráneas)

Al intentar eliminar un usuario que tenía datos asociados (entrenamientos o registros de peso), la base de datos lo impedía por las restricciones de clave foránea. En lugar de forzar el borrado en cascada (que eliminaría datos de forma irreversible), se optó por implementar un **borrado lógico**: en vez de eliminar físicamente los registros, se marcan como inactivos. Esta es la práctica habitual en aplicaciones reales, ya que preserva el histórico y la integridad de los datos. En el caso de los ejercicios, se permite el borrado físico solo cuando no están en uso; si lo están, el sistema avisa y sugiere desactivarlos.

Error de CORS entre frontend y backend

Durante el desarrollo surgió un error de **CORS** (*Cross-Origin Resource Sharing*) que bloqueaba las peticiones del frontend al backend, ya que se ejecutan en puertos distintos. El problema se agravaba porque el servidor de desarrollo del frontend cambiaba de puerto automáticamente. Se solucionó configurando en Spring Security los orígenes permitidos mediante un patrón que admite cualquier puerto de *localhost* durante el desarrollo.

Recuperación de contraseña sin servidor de correo

Se quería ofrecer una forma de recuperar la contraseña, pero implementar el envío de correos electrónicos requería una infraestructura compleja para un proyecto

académico. Como solución, se implementó una recuperación en dos pasos basada en la verificación de la identidad mediante el email y la fecha de nacimiento del usuario, evitando así la dependencia de un servidor de correo.

9.Desarrollo de las pruebas

9.1.Pruebas de código

En este proyecto no se han implementado pruebas de código automatizadas mediante frameworks como JUnit. Las pruebas se han realizado de forma **manual y funcional**, comprobando directamente el comportamiento de cada funcionalidad a través de la interfaz de la aplicación y verificando las respuestas del backend.

Para ello se ha utilizado la propia aplicación en ejecución, así como la consola del navegador y los registros (*logs*) del servidor, que permitían observar las peticiones, las consultas a la base de datos y los posibles errores. Igualmente, se ha consultado directamente la base de datos para verificar que los datos se guardaban correctamente (por ejemplo, comprobando que las contraseñas se almacenaban cifradas).

Se es consciente de que la incorporación de pruebas automatizadas con JUnit en el backend sería una mejora importante, ya que permitiría comprobar de forma rápida y repetible que los cambios en el código no rompen funcionalidades existentes. Por ello, esta tarea se ha recogido como una de las **futuras ampliaciones** del proyecto.

9.2 Pruebas de usabilidad

Para evaluar la usabilidad de la aplicación, se realizó una prueba con una **usuario real ajena al desarrollo** y sin conocimientos técnicos de programación. El objetivo era comprobar si una persona sin experiencia previa con la aplicación era capaz de utilizar sus funciones principales y conocer su impresión sobre el diseño y la facilidad de uso.

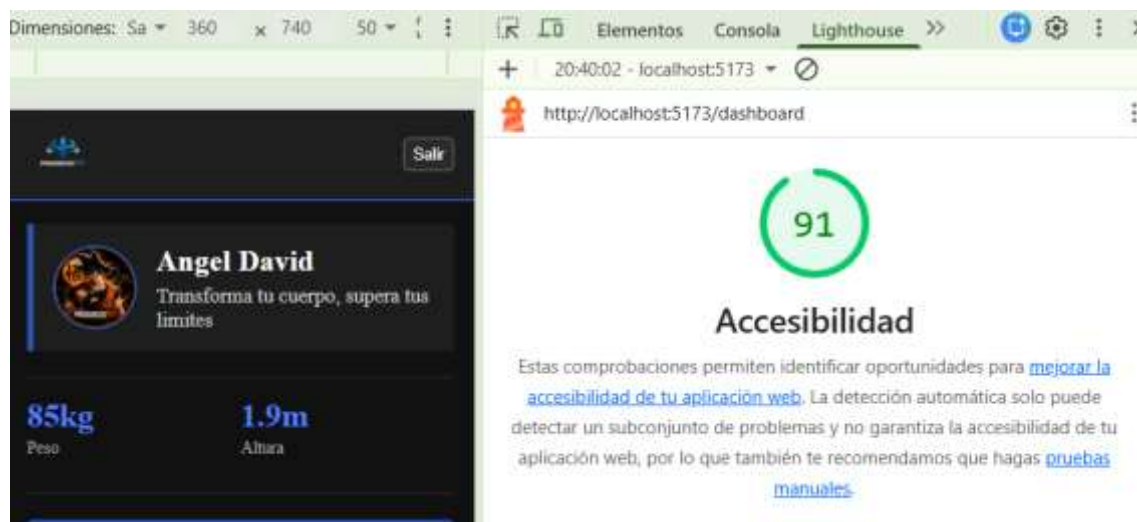
Se le pidió que realizara las tareas más habituales de la aplicación (registrarse, crear un entrenamiento y consultar su progreso), observando si era capaz de completarlas sin ayuda, y se recogieron sus comentarios.

Los resultados de la prueba fueron útiles y dieron lugar a cambios reales en la aplicación:

- **Diseño visual:** en un primer momento, el diseño de la interfaz no le resultó atractivo. A partir de este comentario, se rediseñó el aspecto visual de la aplicación, mejorando la paleta de colores y la presentación de los elementos hasta llegar al diseño actual.
- **Aspectos mejor valorados:** destacó positivamente la selección de avatares y, sobre todo, las gráficas de estadísticas, que le parecieron la parte más visual y atractiva de la aplicación.

Esta prueba, aunque informal y con un único usuario, resultó valiosa, ya que aportó una visión externa que permitió detectar que el diseño inicial no era del todo satisfactorio y mejorarlo. Confirmó además que las funciones principales se podían utilizar sin necesidad de explicaciones previas, lo que indica un buen nivel de usabilidad. Como mejora futura, sería interesante ampliar estas pruebas a un grupo mayor de usuarios de distintos perfiles.

9.3 Pruebas de accesibilidad



Para evaluar la accesibilidad de la aplicación se utilizó la herramienta automática **Lighthouse**, integrada en el navegador Google Chrome, que analiza una página web y genera una puntuación junto con una lista de aspectos de accesibilidad a mejorar. El análisis se realizó sobre la pantalla principal de la aplicación (dashboard) en su versión móvil.

El resultado fue una **puntuación de 91 sobre 100**, que se sitúa en el rango considerado bueno por la herramienta. La aplicación superó la mayoría de las comprobaciones (botones con nombres accesibles, documento con idioma definido, áreas táctiles de tamaño adecuado, orden correcto de los encabezados, etc.). No obstante, Lighthouse detectó **dos aspectos a mejorar**:

- **Contraste de color insuficiente:** algunos textos secundarios, como las fechas de los entrenamientos (en color gris claro sobre fondo oscuro), no alcanzan la relación de contraste mínima recomendada, lo que puede dificultar su lectura a algunos usuarios.
- **Falta de texto alternativo en iconos:** los iconos de la barra de navegación inferior no disponen del atributo alt con texto alternativo, necesario para que los lectores de pantalla puedan describirlos a usuarios con discapacidad visual.

Estos hallazgos resultan útiles porque señalan mejoras concretas y sencillas de aplicar: aumentar el contraste de los textos secundarios y añadir textos alternativos a los iconos. Ambas correcciones se han recogido como mejoras para aumentar aún más el nivel de accesibilidad. En conjunto, una puntuación de 91 sobre 100 indica una buena base de accesibilidad para una primera versión de la aplicación

10. Manuales del sistema

10.1. Manual de despliegue

En este manual se describen los pasos necesarios para poner en funcionamiento la aplicación ProgressFit. El despliegue se ha simplificado mediante **Docker** y **Docker Compose**, de modo que no es necesario instalar manualmente Java, Node, MySQL ni configurar nada por separado: con un único comando se levantan automáticamente los tres servicios que componen la aplicación (base de datos, backend y frontend), cada uno en su propio contenedor.

Requisitos previos

Antes de empezar es necesario tener instalado en el equipo:

- **Docker** (versión 20 o superior).
- **Docker Compose** (versión 2 o superior).
- **Git**, para clonar el repositorio.

Se puede comprobar que Docker está instalado con los comandos `docker --version` y `docker compose version`.

Pasos para ejecutar el proyecto

1. Clonar el repositorio:

```
git clone https://github.com/AngelD87/ProgressFit.git
cd ProgressFit
```

2. Arrancar el proyecto con Docker Compose:

```
docker compose up --build
```

Este comando, de forma automática, descarga la imagen de MySQL 8, compila el backend de Spring Boot, compila el frontend de React y levanta los tres contenedores. La primera vez puede tardar entre 3 y 5 minutos.

3. Acceder a la aplicación: una vez que el backend ha arrancado correctamente (aparece el mensaje "*Started ProgressfitApplication*" en la terminal), se abre el navegador en la dirección <http://localhost>.

Puertos utilizados

Puertos utilizados	
Servicio	Puerto
Frontend	80
Backend	8080
MySQL	3307

Credenciales de prueba

Se puede crear una cuenta nueva desde la propia aplicación, o utilizar las siguientes credenciales de prueba:

Credenciales de prueba		
Rol	Email	Contraseña
Usuario	test@hotmail.es	12345
Administrador	admin@hotmail.es	12345678

Detener el proyecto

Para detener todos los contenedores se utiliza docker compose down. Si además se quieren eliminar los datos almacenados en la base de datos, se usa docker compose down -v.

Arquitectura del despliegue

El despliegue se compone de tres contenedores definidos en el archivo `docker-compose.yml`:

- **Contenedor de MySQL:** ejecuta la base de datos. Utiliza un *volumen* de Docker para que los datos persistan aunque el contenedor se reinicie. Incluye un *healthcheck* que comprueba que la base de datos está lista antes de arrancar el backend.
- **Contenedor del backend:** compila y ejecuta la aplicación Spring Boot. Recibe la configuración de conexión a la base de datos mediante variables de entorno y espera (`depends_on`) a que MySQL esté disponible antes de arrancar.
- **Contenedor del frontend:** se construye en dos fases (*multi-stage build*): primero compila la aplicación React con Node, y después sirve los archivos resultantes mediante un servidor **Nginx** ligero. Esta técnica reduce el tamaño final de la imagen, ya que la imagen de producción solo contiene Nginx y los archivos compilados, no las herramientas de desarrollo.

```
ProgressFit/
├── backend/           ← API REST con Spring Boot
│   └── Dockerfile
├── frontend/         ← Interfaz con React + Vite
│   ├── Dockerfile
│   └── nginx.conf
├── docker-compose.yml
└── despliegue.md
```

10.2. Manual de usuario

Este manual explica cómo utilizar la aplicación ProgressFit desde el punto de vista del usuario, describiendo paso a paso sus funciones principales.

Registro e inicio de sesión

Al abrir la aplicación por primera vez, el usuario debe **crear una cuenta**. Pulsando en "Regístrate", se accede a un formulario dividido en varios pasos en el que se introducen los datos de la cuenta (nombre, email y contraseña), los datos personales (fecha de nacimiento y sexo) y las medidas (altura, peso y objetivo). Una vez registrado, puede **iniciar sesión** introduciendo su email y contraseña en la pantalla de acceso.

Si el usuario olvida su contraseña, puede pulsar en "¿Olvidaste tu contraseña?" y recuperar el acceso verificando su identidad mediante su email y su fecha de nacimiento, tras lo cual podrá establecer una nueva contraseña.

Pantalla principal (Inicio)

Tras iniciar sesión, el usuario accede a la pantalla principal, donde se muestra un resumen con su nombre, su peso y altura actuales, y el listado de sus entrenamientos. Desde aquí puede crear un nuevo entrenamiento o acceder al detalle de los ya registrados. En la parte inferior se encuentra la barra de navegación, que permite moverse entre las distintas secciones: Inicio, Estadísticas, Progreso y Perfil.

Registrar un entrenamiento

Para registrar un entrenamiento, se pulsa en "Nuevo entrenamiento". A continuación, el usuario indica el nombre del entrenamiento y añade los ejercicios que va a realizar, seleccionándolos del catálogo organizado por grupos musculares. Para cada ejercicio puede registrar sus series, indicando el número de repeticiones, el peso utilizado y el RIR (repeticiones en reserva). Al finalizar, puede indicar una valoración y la fatiga percibida de la sesión. El entrenamiento queda guardado en su historial.

Consultar el detalle de un entrenamiento

Desde la pantalla principal, al pulsar sobre un entrenamiento del listado, se accede a su detalle, donde se pueden ver todos los ejercicios y sus series. Desde aquí el usuario puede editar o eliminar las series si necesita corregir algún dato.

Seguimiento del peso corporal

El usuario puede registrar su peso corporal de forma periódica. Cada registro queda almacenado con su fecha, lo que permite ver la evolución del peso a lo

largo del tiempo. Solo se permite un registro de peso por día; si se registra otro el mismo día, se actualiza el anterior.

Página de progreso

En la sección "Progreso", el usuario puede consultar indicadores calculados a partir de sus datos: su **Índice de Masa Corporal (IMC)**, una **estimación de las calorías diarias recomendadas** según su objetivo y nivel de actividad, una barra que muestra el avance hacia su objetivo de peso, y una gráfica con la evolución de su peso corporal.

Página de estadísticas

En la sección "Estadísticas" se muestran varias gráficas que reflejan la evolución del usuario: la progresión del peso levantado (que se puede filtrar por músculo y ejercicio), la fatiga, la frecuencia semanal de entrenamientos y la valoración a lo largo del tiempo. Estas gráficas permiten al usuario comprobar de forma visual si está mejorando.

Perfil de usuario

En la sección "Perfil", el usuario puede consultar y modificar sus datos personales (nombre, altura, fecha de nacimiento, sexo), sus objetivos (peso objetivo y nivel de actividad) y elegir un avatar. También puede registrar su peso corporal desde aquí.

Funciones del administrador

Los usuarios con rol de administrador disponen de un **panel de administración** adicional, desde el que pueden gestionar los usuarios de la aplicación (cambiar su rol y activarlos o desactivarlos), así como administrar el catálogo de grupos musculares y de ejercicios (crear, editar, activar/desactivar y eliminar).

11. Conclusiones

11.1. Desarrollo del proyecto

El desarrollo de ProgressFit ha supuesto la realización de un proyecto completo de aplicación web *full-stack*, abarcando todas las fases: el análisis, el diseño, la implementación del backend y del frontend, el diseño de la base de datos, las pruebas y el despliegue. El resultado es una aplicación funcional que cumple con el objetivo general planteado: ofrecer una herramienta para registrar entrenamientos y hacer un seguimiento del progreso físico, sustituyendo el método tradicional en papel.

En cuanto a los objetivos específicos definidos al inicio, se han cumplido en su totalidad: la aplicación dispone de registro e inicio de sesión con autenticación segura, permite crear entrenamientos con sus ejercicios y series, ofrece un catálogo organizado por grupos musculares, registra el peso corporal, calcula indicadores como el IMC y las calorías, muestra estadísticas mediante gráficas, incluye un perfil de usuario, un panel de administración y un sistema de recuperación de contraseña, y se ha diseñado pensando en su uso desde el móvil.

A nivel personal, este proyecto ha sido una experiencia de aprendizaje muy valiosa. Al comenzar, partía de una versión inicial desarrollada con HTML, CSS y JavaScript puro, y uno de los mayores retos fue tomar la decisión de **migrar todo el frontend a React**. Aunque supuso rehacer gran parte del trabajo, esta migración me permitió aprender una tecnología ampliamente utilizada en el mercado y comprender en la práctica las ventajas de organizar el código en componentes. Del mismo modo, el desarrollo del backend con Spring Boot me ha permitido afianzar conceptos como las API REST, la seguridad con JWT, el mapeo objeto-relacional con JPA o la separación en capas.

Durante el desarrollo surgieron diversas dificultades que tuve que resolver, como el cifrado seguro de las contraseñas, la gestión del borrado de datos relacionados mediante borrado lógico, o un error de CORS que impedía la comunicación entre el frontend y el backend. Enfrentarme a estos problemas y encontrar soluciones razonadas ha sido, probablemente, la parte más formativa del proyecto, ya que refleja el tipo de situaciones reales a las que se enfrenta un desarrollador.

También he aprendido a gestionar un proyecto de cierta envergadura yo solo, organizando el tiempo (compaginándolo con un trabajo), utilizando control de versiones con Git y GitHub, y contenedorizando la aplicación con Docker para facilitar su despliegue.

Como valoración final, considero que el resultado es satisfactorio y que el proyecto cumple con lo que me propuse. Soy consciente de que la aplicación tiene aspectos mejorables y funcionalidades que se podrían añadir, que se recogen en el apartado de futuras ampliaciones, pero la base construida es sólida y funcional. Por encima del resultado concreto, el mayor valor de este proyecto ha sido todo lo aprendido durante su desarrollo, que me ha permitido poner en práctica de forma conjunta las competencias adquiridas a lo largo del ciclo.

11.2 Futuras ampliaciones

ProgressFit es una aplicación funcional, pero como todo proyecto de software, tiene margen de mejora y posibilidades de ampliación. A continuación, se recogen las principales líneas de trabajo futuro, identificadas durante el desarrollo y las pruebas.

Mejoras técnicas y de calidad

- **Pruebas automatizadas con JUnit:** incorporar pruebas unitarias y de integración en el backend para comprobar de forma automática y repetible que los cambios no rompen funcionalidades existentes, complementando las pruebas manuales realizadas.
- **Mejoras de accesibilidad:** corregir los aspectos detectados con la herramienta Lighthouse, principalmente aumentar el contraste de algunos textos secundarios (como las fechas de los entrenamientos) y añadir textos alternativos (alt) a los iconos de navegación, para mejorar la experiencia de usuarios con lectores de pantalla.
- **Página de error 404:** añadir una página personalizada para las rutas que no existen, mejorando la experiencia de navegación.

Nuevas funcionalidades

- **Rol de entrenador funcional:** desarrollar las funciones del rol de entrenador (actualmente contemplado en el modelo de datos), permitiéndole, por ejemplo, asignar rutinas o hacer seguimiento de otros usuarios.
- **Calculadora de calorías de las comidas:** ampliar el apartado nutricional permitiendo al usuario registrar las calorías de los alimentos que consume, complementando la estimación de calorías diarias recomendadas.

- **Funciones sociales:** incorporar la posibilidad de seguir a otros usuarios, compartir entrenamientos o comparar progresos, una característica presente en aplicaciones de la competencia.
- **Rutinas predefinidas y plantillas:** ofrecer rutinas ya creadas que el usuario pueda usar como punto de partida, así como la posibilidad de guardar sus propias rutinas como plantillas para reutilizarlas.
- **Biblioteca de ejercicios ampliada:** aumentar el catálogo de ejercicios e incluir imágenes o vídeos demostrativos de cada uno.

Despliegue y distribución

- **Puesta en producción:** desplegar la aplicación en un servidor en la nube para que sea accesible públicamente, junto con un dominio propio.
- **Aplicación móvil nativa:** dado que el uso principal es desde el móvil, una posible evolución sería desarrollar una aplicación nativa o convertir la web en una *Progressive Web App* (PWA) para mejorar la experiencia en el teléfono.

12. Bibliografía

Durante el desarrollo del proyecto se han consultado principalmente las documentaciones oficiales de las tecnologías utilizadas, así como diversos recursos de aprendizaje y comunidades de desarrolladores.

Documentación oficial

- Spring Boot. Documentación oficial. <https://spring.io/projects/spring-boot>
- Spring Security. Documentación oficial. <https://spring.io/projects/spring-security>
- React. Documentación oficial. <https://react.dev>
- Vite. Documentación oficial. <https://vitejs.dev>
- React Router. Documentación oficial. <https://reactrouter.com>
- Axios. Documentación oficial. <https://axios-http.com>
- Recharts. Documentación oficial. <https://recharts.org>
- MySQL. Documentación oficial. <https://dev.mysql.com/doc>

- Hibernate ORM. Documentación oficial. <https://hibernate.org/orm/documentation>
- Docker. Documentación oficial. <https://docs.docker.com>
- JSON Web Tokens (JWT). <https://jwt.io>

Recursos de aprendizaje y consulta

- MDN Web Docs (Mozilla). Referencia de HTML, CSS y JavaScript. <https://developer.mozilla.org>
- Stack Overflow. Comunidad de desarrolladores. <https://stackoverflow.com>
- Baeldung. Tutoriales sobre Spring y Java. <https://www.baeldung.com>

Fórmulas y referencias

- Fórmula de Mifflin-St Jeor para el cálculo del metabolismo basal y las calorías diarias (TDEE), utilizada en la página de progreso.